

Documentation de Déploiement - M-Atici CRM

1. Introduction

Ce document détaille la stratégie de déploiement mise en place pour l'application M-Atici CRM. L'objectif est de garantir un passage en production fluide, sécurisé, et automatisé via l'approche CI/CD (Intégration Continue et Déploiement Continu).

2. Architecture de l'Infrastructure

L'application est déployable sur n'importe quel environnement Linux (notamment un VPS Cloud comme OVHcloud) grâce à sa conteneurisation complète.

L'écosystème repose sur 3 services majeurs :

- **Frontend** : Application React (distribuée statiquement ou via un serveur Nginx/Caddy).
- **Backend** : API REST Node.js / Express.
- **Base de données** : PostgreSQL 15, couplé avec l'ORM Prisma pour les migrations de schémas.

L'orchestration est gérée par **Docker Compose**.

3. Configuration Docker

Un fichier `docker-compose.yml` (ainsi qu'une variante `docker-compose.staging.yml` pour la pré-production) orchestre les services.

Avantages de cette approche :

- Isolation stricte des services.
- Reproductibilité entre l'environnement de développement et de production.
- Gestion centralisée des variables d'environnement (`.env`).

Extrait de configuration (Backend) :

```
backend:
  build: ./backend
  restart: always
  environment:
    - DATABASE_URL=${DATABASE_URL}
    - JWT_SECRET=${JWT_SECRET}
  ports:
    - "3000:3000"
  depends_on:
    - db
```

4. Pipeline CI/CD (GitHub Actions)

Pour éviter de déployer du code défectueux, un pipeline d'Intégration Continue (CI) a été mis en place sur le dépôt Git.

À chaque validation de code (`git push`) sur la branche principale :

1. **Build** : Le serveur GitHub Actions récupère le code et installe les dépendances Node.js.
2. **Tests** : La suite de tests unitaires (Jest) est exécutée automatiquement.
3. **Validation** : Si un seul test échoue, le processus s'arrête, bloquant ainsi le déploiement d'une régression en production.

(Le fichier de configuration se trouve dans `.github/workflows/main.yml`).

5. Script de Déploiement

Le déploiement proprement dit s'effectue via un script bash (`deploy-staging.sh`) exécuté sur le serveur cible. Ce script automatise les tâches d'exploitation courantes :

1. Mise à jour des sources via Git.
2. Reconstruction des images Docker (`docker-compose build`).
3. Démarrage des conteneurs (`docker-compose up -d`).
4. Exécution des migrations de base de données en production via Prisma (`npm run prisma migrate deploy`).

6. Sécurité et Bonnes Pratiques

- **Secrets** : Les variables sensibles (Mots de passe, clés JWT) ne sont pas versionnées sur Git. Elles sont injectées directement sur le serveur via le fichier `.env` .

- **Certificats SSL** : Le trafic entrant en production est chiffré (HTTPS), généralement géré via un reverse-proxy (Caddy ou Nginx) qui renouvelle automatiquement les certificats Let's Encrypt.
- **Persistance** : Les données de PostgreSQL sont attachées à des Volumes Docker persistants pour prévenir toute perte de données lors du redémarrage des conteneurs.